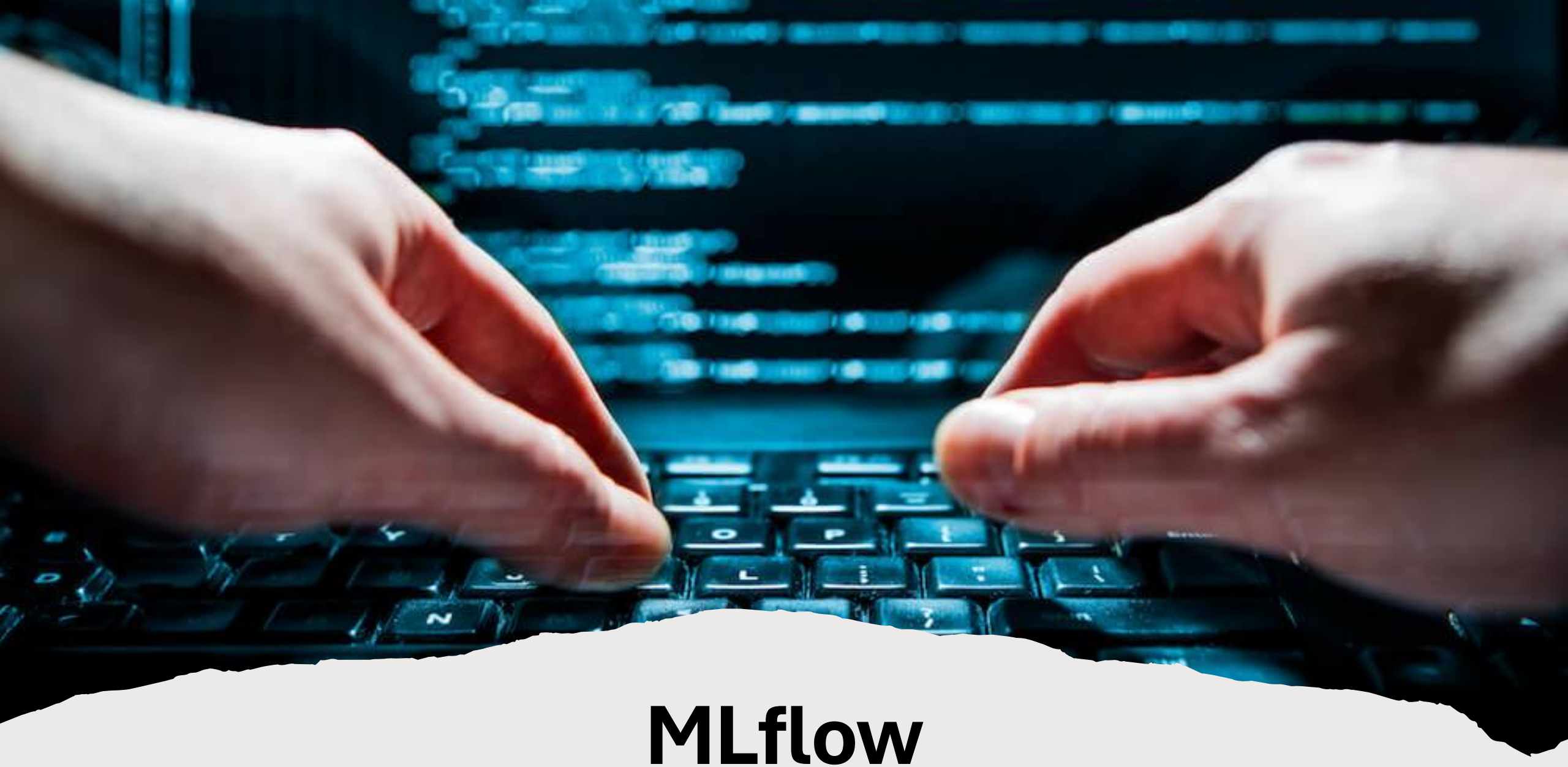


MLflow

Alejandro Morales Hernandez

02-12-2025

Deliver production-ready AI with MLflow

Main functionalities

- Tracks experiment parameters, metrics, and artifacts in a central location
- Packages code and environments for reproducible runs
- Provides model registry with versioning, stage transitions, and annotations
- Supports deployment to multiple targets such as local servers, cloud platforms, and containers
- Offers a unified API for many machine learning libraries
- Enables reproducible pipelines through MLflow Projects

Set up MLflow

```
C:\WINDOWS\system32\cmd.
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aleja>mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
```

Access MLflow UI

```python
# Print connection information
print(f"MLflow Tracking URI: {mlflow.get_tracking_uri()}")
print(f"Active Experiment: {mlflow.get_experiment_by_name('my-first-experiment')}")

# Test logging
with mlflow.start_run():
    mlflow.log_param( key: "test_param",  value: "test_value")
    print("✓ Successfully connected to MLflow!")
```

Verify Your Connection

Configure Tracking

```python
main.py
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("my-first-experiment")
```

Different backends supported.
Recommend dataset

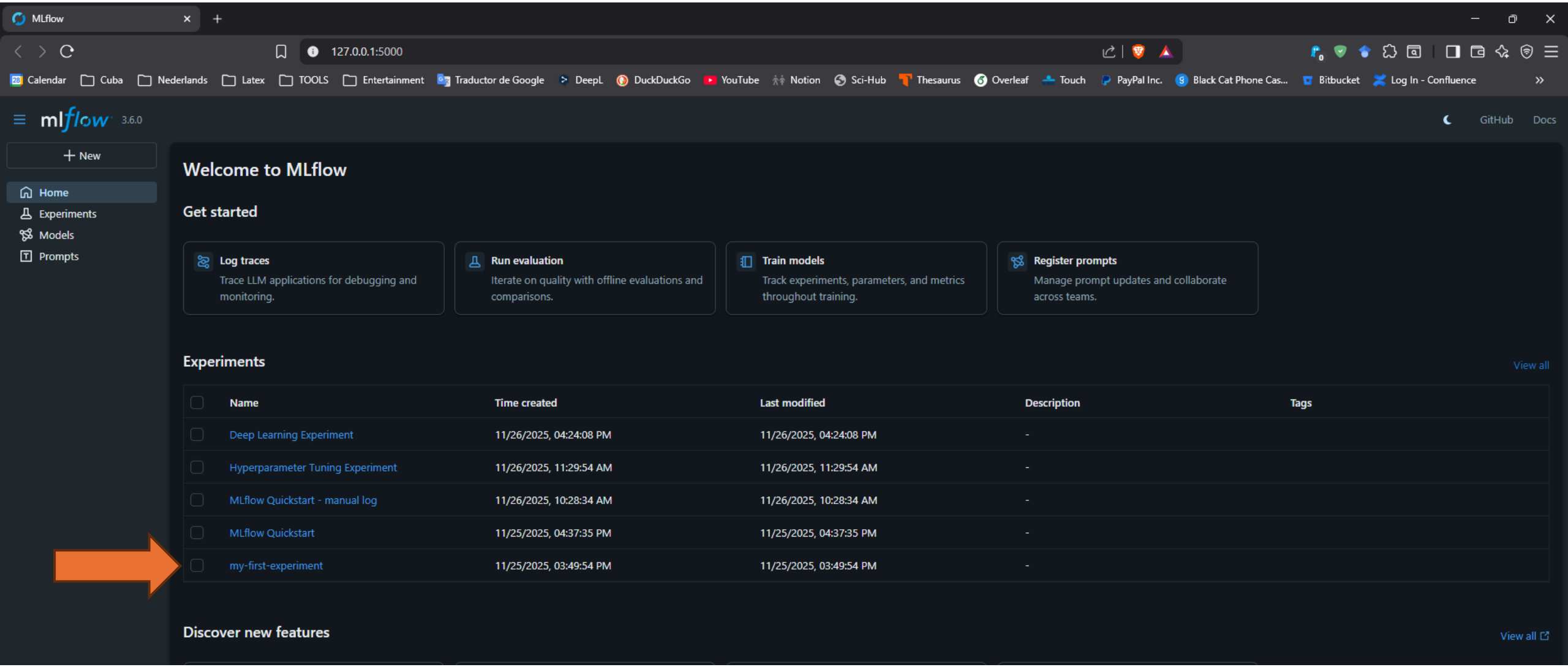
Prepare an experiment

Install MLflow in your project

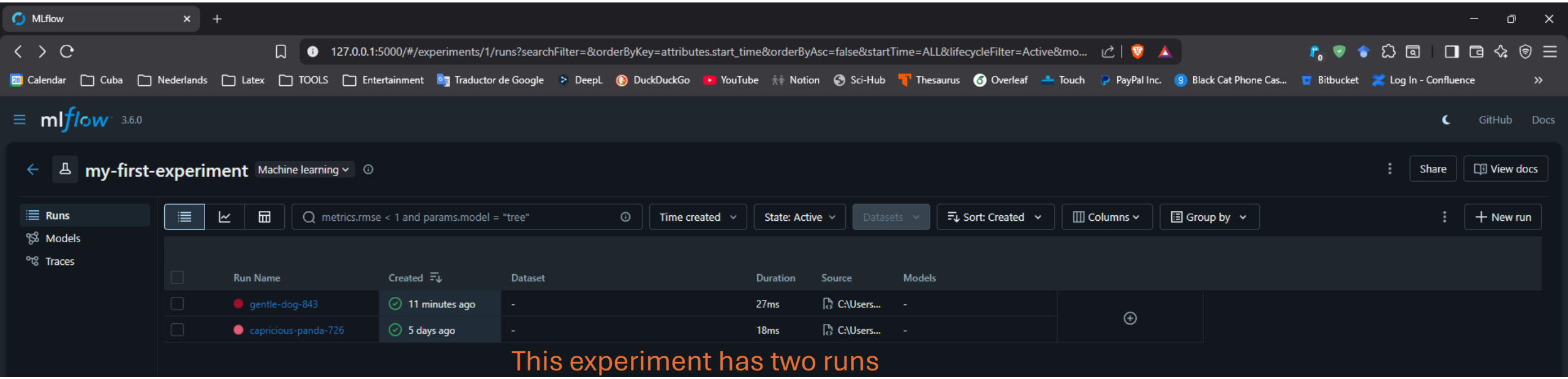
```
C:\WINDOWS\system32\cmd.
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aleja>pip install --upgrade "mlflow>=3.1"
```

Connect Your Development Environment to MLflow

Set up MLflow

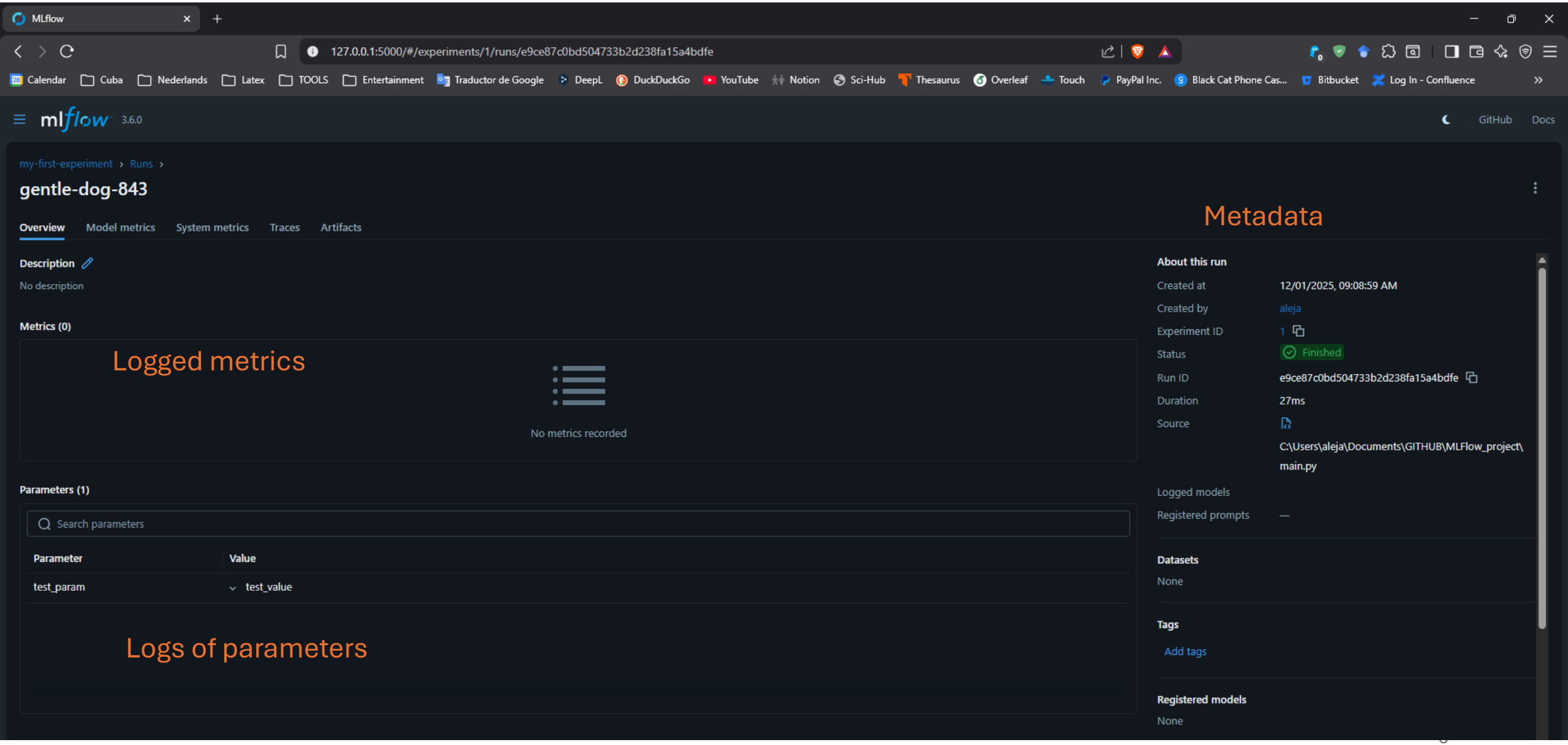


Set up MLflow



This experiment has two runs

Set up MLflow



Autologging with MLflow

MLflow Tracking Quickstart

quick_starting.py

```python
import mlflow

# Step 1. Set up connection
mlflow.set_tracking_uri("sqlite:///mlflow.db")

# Step 2. Add experiment
mlflow.set_experiment("MLflow Quickstart")

# Step 3. Prepare training data
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load the Iris dataset
X, y = datasets.load_iris(return_X_y=True)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Define the model hyperparameters
> params = {...}

# Step 4. Train a model with MLflow Autologging

# Enable autologging for scikit-learn
mlflow.sklearn.autolog()

# Just train the model normally
lr = LogisticRegression(**params)
lr.fit(X_train, y_train)
```

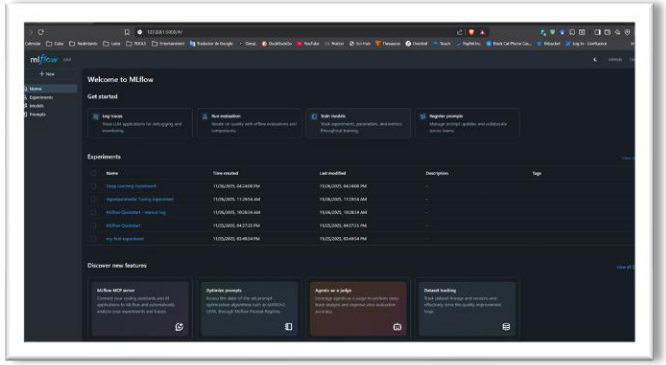
Set the backend

Specify the experiment

Your code

Enable autologging for scikit-learn

Train your model



Log a model and metadata manually

MLflow Tracking Quickstart

Specify the backend and set the experiment

```python
# Start an MLflow run
with mlflow.start_run():
    # Log the hyperparameters
    mlflow.log_params(params)

    # Train the model
    lr = LogisticRegression(**params)
    lr.fit(X_train, y_train)

    # Log the model and keep the MLFlow info to retrieve the model later
    model_info = mlflow.sklearn.log_model(sk_model=lr, name="iris_model")

    # Predict on the test set, compute and log the loss metric
    y_pred = lr.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    mlflow.log_metric( key: "accuracy", accuracy)

    # Optional: Set a tag that we can use to remind ourselves what this run was for
    mlflow.set_tag( key: "Training Info",  value: "Basic LR model for iris data")

# Step 5 - Load the model back for inference.

# Load the model back for predictions as a generic Python Function model
loaded_model = mlflow.pyfunc.load_model(model_info.model_uri)

predictions = loaded_model.predict(X_test)
```

Start a run of the experiment. Use the parameter "*run_name*" to specify a custom name

Log model's parameters

Log the model

Log model's metrics

Add a tag for easy identification

Recover a model

Hyperparameter tuning

Tracking Hyperparameter Tuning with MLflow

Specify the backend and set the experiment

```python
def objective(trial):
    # Setting nested=True will create a child run under the parent run.
    with mlflow.start_run(nested=True, run_name=f"trial_{trial.number}") as child_run:
        # All the information logged here is going to be attached to the trails in MLFlow
        rf_max_depth = trial.suggest_int("rf_max_depth", 2, 32)
        rf_n_estimators = trial.suggest_int("rf_n_estimators", 50, 300, step=10)
        rf_max_features = trial.suggest_float("rf_max_features", 0.2, 1.0)
        params = {
            "max_depth": rf_max_depth,
            "n_estimators": rf_n_estimators,
            "max_features": rf_max_features,
        }
        # Log current trial's parameters
        mlflow.log_params(params)

        regressor_obj = sklearn.ensemble.RandomForestRegressor(**params)
        regressor_obj.fit(X_train, y_train)

        y_pred = regressor_obj.predict(X_val)
        error = sklearn.metrics.mean_squared_error(y_val, y_pred)
        # Log current trial's error metric
        mlflow.log_metrics({"error": error})

        # Log the model file
        mlflow.sklearn.log_model(regressor_obj, name="model")
        # Make it easy to retrieve the best-performing child run later
        trial.set_user_attr("run_id", child_run.info.run_id)
        return error
```

Log model's parameters

Log model's metrics

Log model

Associate each HP trial to a run in MLflow

In HPO, a study is conformed by several trials that represent an HP evaluation

Add the run's identifier to each trial (specific to HPO library)

Hyperparameter tuning

Tracking Hyperparameter Tuning with MLflow

```python
with mlflow.start_run(run_name="study") as run:
    # All the information logged here is going to be attached to the study in MLFlow
    # Log the experiment settings
    n_trials = 5
    mlflow.log_param( key: "n_trials", n_trials)

    study = optuna.create_study(direction="minimize")
    study.optimize(objective, n_trials=n_trials)

    # Log the best trial and its run ID
    mlflow.log_params(study.best_trial.params)
    mlflow.log_metrics({"best_error": study.best_value})
    if best_run_id := study.best_trial.user_attrs.get("run_id"):
        mlflow.log_param( key: "best_child_run_id", best_run_id)
        best_model_uri = f"runs:/{best_run_id}/model"
        print("Best model URI:", best_model_uri)

# Register the best model using the model URI
rm = mlflow.register_model(
    model_uri=best_model_uri,
    name="best-hpo-method",
)

loaded_model = mlflow.pyfunc.load_model(best_model_uri)
```

Create a parent run that contains all child runs for different trials

Log specific parameters of this run

Specific HPO library code

Because we saved the run's ID on each trial, we can recover it by using specific functionalities of the HPO library

Register the model for production

Recover the best model

What happens if we didn't run the code in our computer?

Recovering models

- A run is identified in an experiment as **runs:/<ID>/<artifact_path>**, where *ID* is the URI (or name) of the run in Mlflow, and *artifact_path* is the "folder" name you used when you logged the model, often just **model**.
- A registered model is usually identified in a run as **models:/<ID>/<register>**, where *ID* is the registered model name and *register* is either a stage such as Staging or Production, or a numeric version like 1 or 3.

```python
# Variable to specify
experiment = "Hyperparameter Tuning Experiment"
parent_run_name = "study"


# Step 0. Set up connection
client = MlflowClient(tracking_uri="sqlite:///mlflow.db")

# Step 1: Find the parent run (named "study")
experiments = client.search_experiments(filter_string=f"name = '{experiment}'")
experiment_id = experiments[0].experiment_id

parent_runs = client.search_runs(
    experiment_ids=[experiment_id],
    filter_string=f"attributes.run_name = '{parent_run_name}'"
)
```

Minimum information needed to access the whole HPO experiment

Set up a connection to the backend

Find the experiment

Find the parent run with the HPO run

Recovering models

- A run is identified in an experiment as **runs:/<ID>/<artifact_path>**, where *ID* is the URI (or name) of the run in Mlflow, and *artifact_path* is the "folder" name you used when you logged the model, often just **model**.
- A registered model is usually identified in a run as **models:/<ID>/<register>**, where *ID* is the registered model name and *register* is either a stage such as Staging or Production, or a numeric version like 1 or 3.

```python
child_runs = client.search_runs(
    experiment_ids=[experiment_id],
    filter_string=f"tags.mlflow.parentRunId = '{parent_run_id}'"
)

# Step 3: Select the best child run based on the metric "error"
best_child = min(child_runs, key=lambda r: r.data.metrics["error"])
best_run_id = parent_run.data.params["best_child_run_id"]
print("Best child run ID:", best_run_id)
print("Best error:", best_child.data.metrics["error"])

# Step 4: Construct the model URI
best_model_uri = f"runs:/{best_run_id}/model"
print("Best model URI:", best_model_uri)

# Step 5: Set up the experiment and load the best model
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("Hyperparameter Tuning Experiment")

best_model = mlflow.sklearn.load_model(best_model_uri)
```

Find all the trials of the HPO parent run

Find the best trial

Construct the model URI

Specify the backend and set the experiment to work with

Retrieve the best model

Model Signatures and Input Examples

Model Signature - Defines the expected format for model inputs, outputs, and parameters.

Model Input Example - Provides a concrete example of valid model input. This helps developers understand the required data format and validates that your model works correctly.

- **Consistency**: Ensure all model interactions follow the same data format
- **Validation**: Catch data format errors before they reach your model
- **Documentation**: Serve as living documentation for model usage
- **Deployment Safety**: Enable MLflow deployment tools to validate requests automatically
- **UI Integration**: Allow MLflow UI to display clear model requirements

Model Signatures and Input Examples

Model Signature - Defines the expected format for model inputs, outputs, and parameters.

Model Input Example - Provides a concrete example of valid model input. This helps developers understand the required data format and validates that your model works correctly.

```python
import mlflow
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

# Load data and train model
iris = load_iris(as_frame=True)
X = iris.data
y = iris.target
model = RandomForestClassifier().fit(X, y)

with mlflow.start_run():
    # The input example automatically generates a signature
    mlflow.sklearn.log_model(
        model, name="iris_model", input_example=X.iloc[[0]]  # First row as example
    )
```

Register a model and let Mlflow to infer the signature

```python
# Column-based signature (DataFrames)
input_schema = Schema(
    [
        ColSpec("double", "sepal_length"),
        ColSpec("double", "sepal_width"),
        ColSpec("string", "species", required=False),  # Optional field
    ]
)

# Tensor-based signature (NumPy arrays)
input_schema = Schema(
    [TensorSpec(np.dtype(np.float32), (-1, 28, 28, 1))]  # Batch of 28x28 images
)
```

Inferred signature

Model Signatures and Input Examples

Input signature

```python
# Column-based signature (DataFrames)
input_schema = Schema(
    [
        ColSpec("double", "sepal_length"),
        ColSpec("double", "sepal_width"),
        ColSpec("string", "species", required=False),
    ]
)

# Tensor-based signature (NumPy arrays)
input_schema = Schema(
    [TensorSpec(np.dtype(np.float32), (-1, 28, 28, 1))]
)
```

Output signature

```python
# Single prediction column
output_schema = Schema([ColSpec("long", "prediction")])

# Multiple outputs
output_schema = Schema(
    [
        ColSpec("double", "probability"),
        ColSpec("string", "predicted_class"),
        ColSpec("long", "confidence_score"),
    ]
)

# Tensor output
output_schema = Schema(
    [TensorSpec(np.dtype(np.float32), (-1, 10))]  # 10-c
)
```

Param signature

```python
# Define inference parameters
params_schema = ParamSchema(
    [
        ParamSpec("temperature", "double", 0.7),  # Default temperature
        ParamSpec("max_tokens", "long", 100),  # Default max tokens
        ParamSpec("stop_words", "string", [".", "!"], (-1,)),  # List parameter
    ]
)
```

```python
# Use in model signature
signature = ModelSignature(
    inputs=input_schema, outputs=output_schema, params=params_schema
)
```

Model signature

```python
with mlflow.start_run():
    mlflow.sklearn.log_model(model, name="my_model", signature=signature)
```

Log the model with the explicit signature

MLflow Dataset Tracking

- MLflow allows you to track, version, and manage datasets used in training, validation, and evaluation, providing complete lineage from raw data to model predictions.
- You don't share the dataset, you share the metadata that allows you to access the dataset
- Support for local files, cloud storage, database, and URL sources
- The actual CSV or Parquet file (or other data file) is saved in the **artifact store**, which by default is a local folder (mlruns/ in local runs).

```python
import mlflow.data
import pandas as pd

# Load your data
dataset_source_url = "https://raw.githubusercontent.com/mlflow/mlflow/master/tests/datasets/winequality-white.csv"
raw_data = pd.read_csv(dataset_source_url, delimiter=";")

# Create a Dataset object
dataset = mlflow.data.from_pandas(
    raw_data, source=dataset_source_url, name="wine-quality-white", targets="quality"
)

# Log the dataset to an MLflow run
with mlflow.start_run():
    mlflow.log_input(dataset, context="training")
```

Dataset's source

Create the dataset object

Log the dataset

MLflow Model Registry

Centralized model store, set of APIs and a UI designed to collaboratively manage the full lifecycle of a model.

- **Version Control**: The registry automatically tracks versions of each model
- **Model Lineage and Traceability**: Each registered model version is linked to the MLflow run, logged model or notebook that produced it, enabling full reproducibility.
- **Production-Ready Workflows**: Features like model aliases (e.g., @champion) and tags make it easier to manage deployment workflows
- **Governance and Compliance**: With structured metadata, tagging, and role-based access controls (when used with a backend like Databricks or a managed MLflow service), the Model Registry supports governance requirements critical for enterprise-grade ML operations.

MLflow Model Registry

Register a model with MLflow Python APIs

Model Registry Tutorials

```
# Option 1: specify `registered_model_name` parameter when logging a model
mlflow.<flavor>.log_model(..., registered_model_name="<YOUR_MODEL_NAME>")

# Option 2: register a logged model
mlflow.register_model(model_uri="<YOUR_MODEL_URI>", name="<YOUR_MODEL_NAME>")
```

After registering the model, you can load it back with the model's name and version

```
mlflow.<flavor>.load_model("models:/<YOUR_MODEL_NAME>/<YOUR_MODEL_VERSION>")
```

Mlflow flavors

Python Function (python_function)
R Function (crate)
H2O (h2o)
Keras (keras)
PyTorch (pytorch)
Scikit-learn (sklearn)
Spark MLlib (spark)

TensorFlow (tensorflow)
ONNX (onnx)
XGBoost (xgboost)
LightGBM (lightgbm)
CatBoost (catboost)
Spacy(spaCy)
Statsmodels (statsmodels)
Prophet (prophet)

Pmdarima (pmdarima)
John Snow Labs (johnsnowlabs)
Transformers (transformers)
SentenceTransformers (sentence_transformers)

MLflow Model Registry

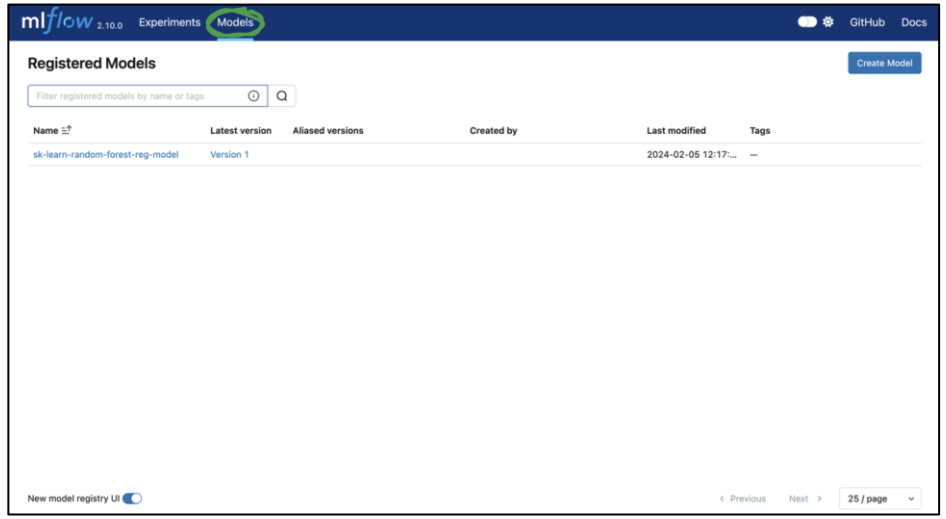
Model Registry Tutorials

```python
# Log the sklearn model and register as version 1
mlflow.sklearn.log_model(
    sk_model=model,
    name="sklearn-model",
    input_example=X_train,
    registered_model_name="sk-learn-random-forest-reg-model",
)
```

```
Successfully registered model 'sk-learn-random-forest-reg-model'.
Created version '1' of model 'sk-learn-random-forest-reg-model'.
```

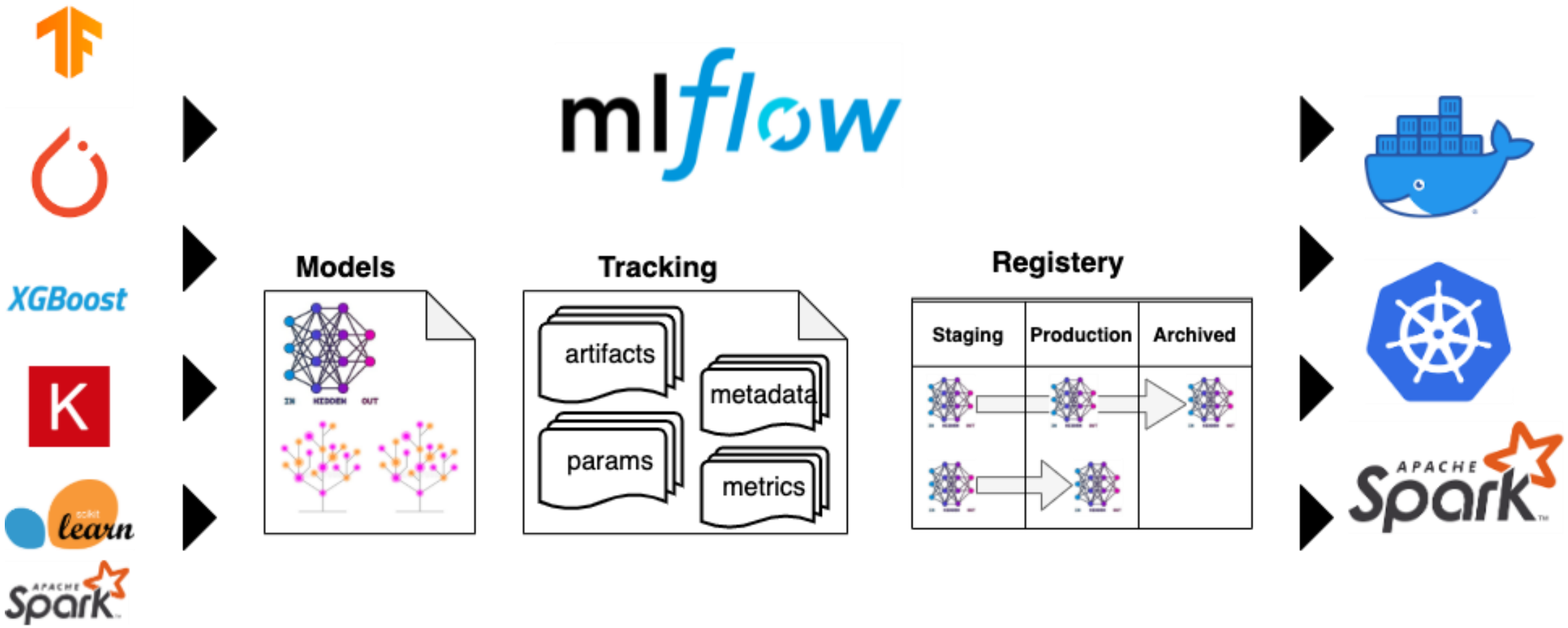
```
mlruns/
├── 0/                                  # Experiment ID
│   ├── bc6dc2a4f38d47b4b0c99d154bbc77ad/ # Run ID
│   │   ├── metrics/
│   │   │   └── mse                     # Example metric file for mean squared error
│   │   ├── artifacts/                  # Artifacts associated with our run
│   │   │   └── sklearn-model/
│   │   │       ├── python_env.yaml
│   │   │       ├── requirements.txt    # Python package requirements
│   │   │       ├── MLmodel             # MLflow model file with model metadata
│   │   │       ├── model.pkl           # Serialized model file
│   │   │       ├── input_example.json
│   │   │       └── conda.yaml
│   │   ├── tags/
│   │   │   ├── mlflow.user
│   │   │   ├── mlflow.source.git.commit
│   │   │   ├── mlflow.runName
│   │   │   ├── mlflow.source.name
│   │   │   ├── mlflow.log-model.history
│   │   │   └── mlflow.source.type
│   │   ├── params/
│   │   │   ├── max_depth
│   │   │   └── random_state
│   │   └── meta.yaml
│   └── meta.yaml
├── models/                             # Model Registry Directory
    ├── sk-learn-random-forest-reg-model/ # Registered model name
    │   ├── version-1/                  # Model version directory
    │   │   └── meta.yaml
    │   └── meta.yaml
```

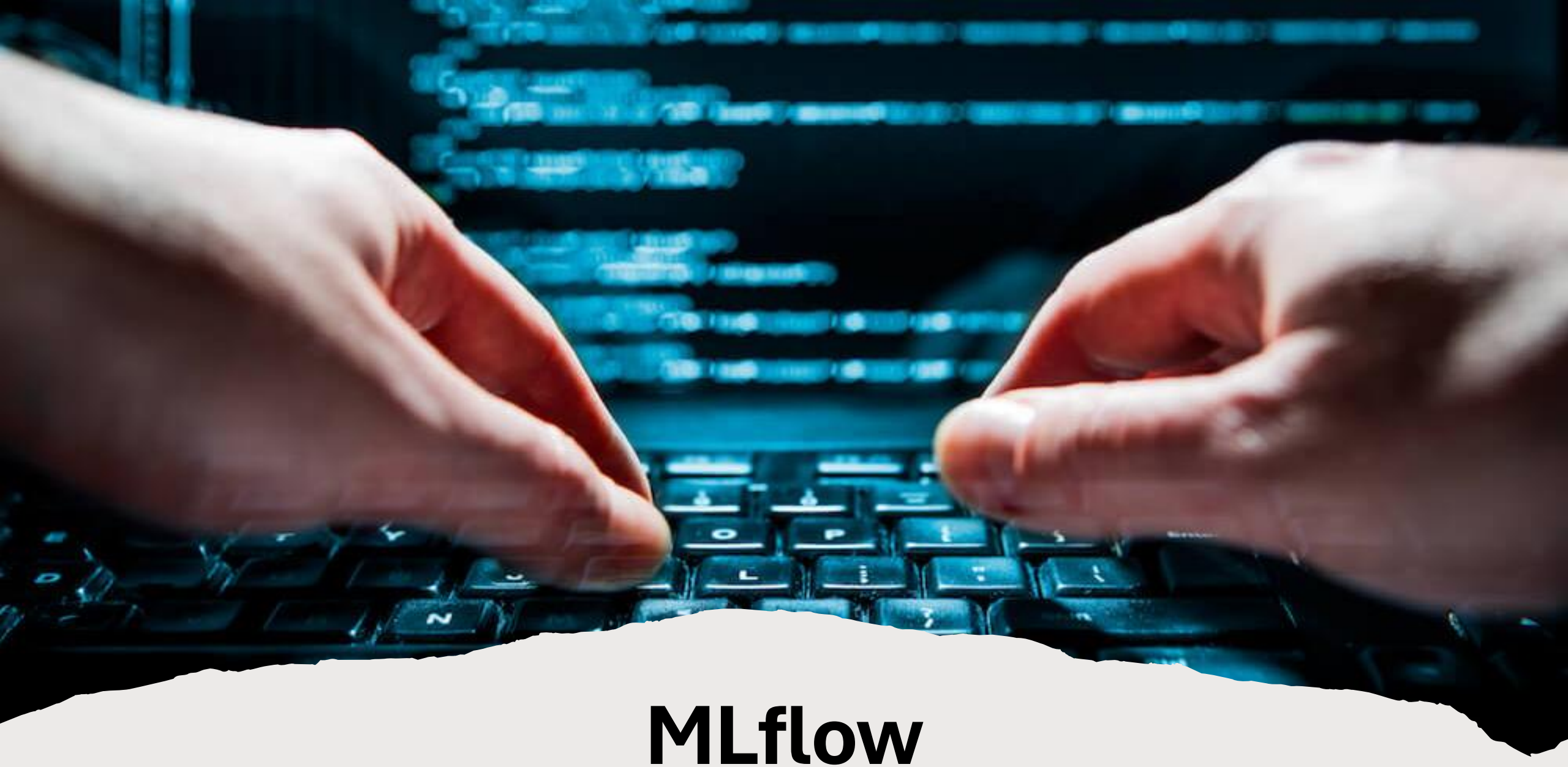
local filesystem as our tracking server and model registry



Start the Tracking Server in the *mlruns* directory

```
mlflow server --host 127.0.0.1 --port 8080
```


mlflow
XGBoost
K
learn
Spark
Models
IN
HIDDEN
OUT
Tracking
artifacts
metadata
params
metrics
Registery
Staging
Production
Archived
APACHE
Spark



MLflow

02-12-2025